

# Hermes Creative Workstation

Investigação tecnológica e arquitetural | 9 de outubro de 2026

## Diagnóstico executivo

A investigação já revela uma distinção fundamental: o software mais interessante para adaptar não é necessariamente aquele que deveria controlar a arquitetura do Hermes Creative Workstation.

Existem duas decisões diferentes:

1. Qual editor oferece a maior quantidade de funcionalidades reutilizáveis agora?
2. Qual arquitetura permite conservar essas funcionalidades, acrescentar outras e manter a edição compartilhada entre humanos e agentes, sem depender permanentemente de um único fornecedor?

Uma solução pode vencer a primeira disputa e perder a segunda.

## Descobertas confirmadas

OpenReel merece prioridade de auditoria.

Seu código público já apresenta separação entre interface, núcleo de edição, aplicações desktop/web e ferramentas para agentes. A documentação descreve 228 ferramentas especializadas e operações sobre timelines, keyframes, composições 2D e cenas 3D editáveis. Isso o torna um candidato especialmente relevante, embora ainda não prove integridade de exportação nem interoperabilidade completa.



GitHub

+1

OpenCut não deve ser escolhido com base na sua arquitetura futura.

O próprio projeto informa que está sendo reescrito. Núcleo Rust, API pública, plugins, MCP e execução headless integram a visão da reescrita, não um conjunto de capacidades cuja prontidão para produção esteja demonstrada. O projeto clássico é uma alternativa separada.



O Hermes Work já possui uma base arquitetural que não deve ser duplicada.

Examinei diretamente `ROADMAP.md`, `ARCHITECTURE.md`, `SOURCE_MATRIX.md`, `HERMES_WORKSTATION_INTELLIGENCE.md` e a documentação da iniciativa criativa no repositório conectado. Já existem contratos para TaskRun, execução, autorização, registro de artefatos, verificação e aprendizagem operacional. A iniciativa criativa está documentada, mas não implementada no runtime.

A hipótese central a testar é utilizar um editor open source existente como ponto de partida, porém conservar sua engine, seu modelo de projeto e suas funcionalidades atrás de contratos de integração, evitando transformá-lo automaticamente no núcleo do Hermes Work.

A seguir, separo o inventário tecnológico, a auditoria dos candidatos e a recomendação de engenharia.

## Parte I — Mapa tecnológico

A pesquisa identificou cinco classes de software que não devem ser confundidas: editores visuais completos, bibliotecas de manipulação, motores de animação, sistemas de processamento de mídia e ferramentas de automação.

### 1. Design 2D

| Tecnologia                            | O que realmente fornece                                                               | Avaliação para o Hermes                                                             |
|---------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Penpot —<br><a href="#">Código</a>    | Editor vetorial completo, documentos estruturados, componentes, colaboração e plugins | Excelente editor especializado; difícil converter seu modelo em NLE/3D universal    |
| Graphite —<br><a href="#">Código</a>  | Editor vetorial/procedural, arquitetura Rust e grafos de operações                    | Arquitetura especialmente interessante para edição não destrutiva                   |
| Fabric.js —<br><a href="#">Código</a> | Objetos Canvas editáveis, transformações e SVG                                        | Boa biblioteca para construir ferramentas visuais; não é aplicativo completo        |
| Konva —<br><a href="#">Código</a>     | Scene graph 2D, eventos, seleção, arraste, transformações e camadas                   | Excelente infraestrutura de canvas; exige ferramentas de editor e timeline próprias |
| Paper.js —<br><a href="#">Código</a>  | Geometria vetorial, paths e operações com curvas                                      | Complemento especializado para edição vetorial                                      |

| Tecnologia                             | O que realmente fornece                            | Avaliação para o Hermes                                             |
|----------------------------------------|----------------------------------------------------|---------------------------------------------------------------------|
| SVG.js —<br><a href="#">Código</a>     | Manipulação programática de SVG                    | Bom para SVG editável e operações determinísticas                   |
| PixiJS —<br><a href="#">Código</a>     | Renderização 2D acelerada por GPU                  | Excelente renderizador, não editor                                  |
| Excalidraw —<br><a href="#">Código</a> | Editor de desenho e diagramas colaborativos        | Útil como referência de interação; limitado para design de produção |
| tldraw —<br><a href="#">Código</a>     | SDK de canvas interativo com ferramentas de edição | Tecnicamente sólido, mas exige licença para uso em produção         |

A ressalva do tldraw é importante: seu SDK atual possui licença própria, diferente de MIT, e requer chave apropriada para produção. Portanto, não é uma dependência permissiva equivalente ao Konva.



tldraw Docs

+1

Descoberta: para um editor que já precisa de timeline, vídeo e camadas 3D, escolher uma biblioteca 2D como núcleo implica desenvolver uma parcela considerável do produto. O fato de Konva ou Fabric.js facilitarem a manipulação de objetos não significa que forneçam infraestrutura de montagem audiovisual.

## 2. Motion graphics

| Tecnologia                    | Capacidades principais                                            | Limitação decisiva                                      |
|-------------------------------|-------------------------------------------------------------------|---------------------------------------------------------|
| <a href="#">Remotion</a>      | Composições em React/TypeScript e renderização de vídeo por frame | Não é um NLE visual completo; licença própria           |
| <a href="#">Revideo</a>       | Geração programática de vídeos e cenas em TypeScript              | Não substitui editor visual de camadas e keyframes      |
| <a href="#">Motion Canvas</a> | Animações vetoriais por código com editor de pré-visualização     | Não pretende substituir um editor de vídeo convencional |
| <a href="#">Theatre.js</a>    | Sequenciamento, propriedades animáveis e edição temporal          | Não fornece, sozinho, um sistema audiovisual completo   |
| <a href="#">Motion</a>        | Animações web e React                                             | Biblioteca de animação, não software autoral completo   |
| <a href="#">Anime.js</a>      | Timeline, easing e animação de propriedades                       | Não fornece persistência de projeto nem exportação NLE  |

| Tecnologia             | Capacidades principais                         | Limitação decisiva                                                 |
|------------------------|------------------------------------------------|--------------------------------------------------------------------|
| <a href="#">GSAP</a>   | Animação avançada DOM/SVG e sequenciamento     | Licença e condições por produto precisam de análise; não é editor  |
| <a href="#">Lottie</a> | Reprodução de animações vetoriais serializadas | Formato de entrega/intercâmbio com subconjunto limitado de efeitos |

O Motion Canvas, por exemplo, tem um editor real, mas ele serve principalmente à autoria por código e à pré-visualização. A própria documentação delimita que não é substituto de um NLE tradicional.



GitHub

+1

O Remotion oferece grande produtividade para vídeos parametrizados, mas sua licença é source-available, não uma licença open source aprovada pela OSI. Em outubro de 2026, indivíduos e organizações elegíveis de até três pessoas podem utilizá-lo gratuitamente sob determinadas condições; a integração comercial e a distribuição exigem avaliação específica.



Make videos programmatically

+1

### 3. Design 3D

| Tecnologia                                                     | Natureza                                          | Adequação                                                                   |
|----------------------------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------|
| <a href="#">Three.js + Editor</a>                              | Engine MIT e editor web                           | Forte compatibilidade com a stack React/Electron                            |
| <a href="#">React Three Fiber</a>                              | Binding React para Three.js                       | Excelente para cenas declarativas e programação por IA                      |
| <a href="#">Babylon.js</a> + <a href="#">Babylon.js Editor</a> | Engine 3D e aplicativo de edição                  | Alternativa particularmente interessante para edição por agentes            |
| <a href="#">PlayCanvas Engine</a>                              | Engine WebGL/WebGPU open source                   | Engine é aberta, mas não equivale a um editor inteiro livre para incorporar |
| <a href="#">Blender</a>                                        | Aplicativo profissional 3D, animação e composição | Muito forte como motor externo controlado por Python                        |

| Tecnologia                | Natureza                         | Adequação                                                     |
|---------------------------|----------------------------------|---------------------------------------------------------------|
| <a href="#">Godot</a>     | Engine e editor 2D/3D            | Útil para cenas interativas, menos natural como núcleo de NLE |
| <a href="#">OpenJSCAD</a> | Geometria e modelagem por código | Extensão para criação procedural, não um editor geral         |

Uma alternativa que merece investigação específica é o Babylon.js Editor. Seu repositório contém código de ações para agentes sobre hierarquias de cena, meshes, materiais, câmeras, luzes, scripts e animações, além de um servidor MCP. Porém, encontrei também o `initializeMcpServer` não ativado no ponto de inicialização principal inspecionado. Isso exige verificar a distribuição concreta antes de considerá-lo pronto para uso automatizado.



GitHub

+1

## 4. Edição de vídeo e infraestrutura de mídia

| Tecnologia                     | Papel real                                                           | Decisão preliminar                                                              |
|--------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <a href="#">OpenReel</a>       | Editor visual com NLE, motion, áudio, 3D e agentes                   | Finalista principal                                                             |
| <a href="#">OpenCut</a>        | Editor em reconstrução arquitetural                                  | Acompanhar; não depender do núcleo Rust ainda                                   |
| <a href="#">MLT Framework</a>  | Framework nativo de timeline, composição e processamento audiovisual | Forte candidato a renderização/NLE especializado                                |
| <a href="#">Shotcut</a>        | Aplicativo NLE maduro sobre MLT                                      | Referência de estabilidade e funcionalidades, difícil incorporação da interface |
| <a href="#">Kdenlive</a>       | NLE avançado baseado em MLT                                          | Aplicativo externo, não componente React                                        |
| <a href="#">FFmpeg</a>         | Decodificação, filtros, codecs, transcodificação e exportação        | Infraestrutura essencial, não editor                                            |
| <a href="#">MediaBunny</a>     | Biblioteca TypeScript de processamento de contêineres de mídia       | Complemento natural de aplicações web                                           |
| <a href="#">OpenTimelineIO</a> | Modelo de intercâmbio de timelines                                   | Útil para importação/exportação; não é documento criativo universal             |

| Tecnologia                  | Papel real                        | Decisão preliminar                                           |
|-----------------------------|-----------------------------------|--------------------------------------------------------------|
| <a href="#">GStreamer</a>   | Framework de pipelines multimídia | Poderoso, com custo de integração maior                      |
| <a href="#">LosslessCut</a> | Corte e remux de mídia            | Ferramenta especializada; não fornece compositor             |
| <a href="#">Editly</a>      | Montagem automatizada de vídeos   | Simples para receitas e exportações; não editor bidirecional |

O OpenTimelineIO é especialmente útil para intercâmbio entre editores, mas não padroniza automaticamente o comportamento de todos os efeitos, shaders, máscaras, tipos de objetos e renderizadores.



GitHub

## 5. Inteligência artificial local

O melhor caminho é integrar modelos como operadores sobre ativos e estruturas editáveis, não transformar cada operação de IA em uma imagem ou vídeo final sem possibilidade de edição.

| Função                                | Candidato                                                    | Saída ideal no Hermes                             |
|---------------------------------------|--------------------------------------------------------------|---------------------------------------------------|
| Remoção de fundo de imagem            | <a href="#">rembg</a>                                        | Máscara/alpha preservados separadamente           |
| Segmentação e acompanhamento em vídeo | <a href="#">SAM 2</a>                                        | Máscara temporal editável                         |
| Visão computacional e tracking        | <a href="#">OpenCV</a>                                       | Trajetória e keyframes aplicáveis                 |
| Transcrição e legendas                | <a href="#">faster-whisper</a> e <a href="#">whisper.cpp</a> | Legendas temporizadas e revisáveis                |
| Separação de stems de áudio           | <a href="#">Demucs</a>                                       | Trilhas de áudio independentes                    |
| Interpolação de frames                | <a href="#">RIFE</a>                                         | Mídia derivada com fonte e parâmetros preservados |
| Rigging e renderização 3D             | <a href="#">Blender Python API</a>                           | Cena/rig/ações e GLB exportável                   |

A compatibilidade local deve ser qualificada no hardware real, especialmente para modelos de vídeo, optical flow e segmentação em sequências longas. Licença do código não equivale à licença dos pesos de IA.

## Parte II — Repositórios e forks: inventário crítico

### Projetos que merecem auditoria aprofundada

| Projeto                                                         | Licença principal                                | Estado observado                                                               | Valor estratégico                   |
|-----------------------------------------------------------------|--------------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------|
| <a href="#">OpenReel</a>                                        | MIT                                              | Código ativo; módulos de motion, NLE, exportação e agentes                     | Altíssimo                           |
| <a href="#">Penpot</a><br><a href="#">Motion — fork girafic</a> | MPL-2.0                                          | Implementação extensa, declaradamente experimental                             | Altíssimo para 2D animado           |
| <a href="#">SynthCut</a>                                        | GPL-3.0                                          | Núcleo NLE e MCP implementados no código; execução não validada nesta pesquisa | Alto para vídeo comandado por IA    |
| <a href="#">Penpot</a>                                          | MPL-2.0                                          | Editor maduro e ativo; MCP oficial                                             | Alto para design colaborativo       |
| <a href="#">Babylon.js Editor</a>                               | Apache-2.0 (verificar artefatos)                 | Editor 3D e código MCP; ativação ainda requer prova                            | Alto para 3D                        |
| <a href="#">Graphite</a>                                        | MIT/Apache-2.0                                   | Evolução ativa; animação tradicional ainda em desenvolvimento                  | Alto como referência arquitetural   |
| <a href="#">OpenCut</a>                                         | MIT                                              | Reescrita estrutural incompleta                                                | Baixo para entrega imediata         |
| <a href="#">Motion Canvas</a>                                   | MIT                                              | Biblioteca e editor especializado                                              | Alto como engine complementar       |
| <a href="#">MLT</a>                                             | LGPL, com componentes sujeitos a outras licenças | Framework nativo maduro                                                        | Alto para processamento audiovisual |

Estado baseado em documentação, metadados e arquivos de código inspecionados em 9/10/2026. Não significa que cada aplicativo tenha sido instalado ou executado nesta pesquisa.

### Investigação dos forks

1. [girafic/penpot](#) — o fork de maior interesse arquitetural

[Abrir branch de animação](#)

A comparação da branch `girafic-penpot-timeline-animation` com o `develop` do próprio fork encontrou 99 commits à frente e 50 atrás. Na comparação com o `develop` oficial do Penpot, ela está 99 commits à frente e 378 atrás. São bases divergentes: esses números não medem apenas o tamanho da funcionalidade de animação.

A análise dos arquivos acrescentados confirmou mudanças em schema, persistência, transformação de camadas, timeline, exportação, interface e testes. Portanto, não é apenas um fork com uma timeline desenhada na interface.

O autor informa, entretanto, que a estrutura ainda pode mudar e recomenda não utilizar arquivos importantes. A funcionalidade é 2D; modelagem 3D, path morphing e tipografia animada por letra ainda não estão completamente atendidos.



Penpot Community

2. `WASasquatch/openreel-video-comfyui` — integração real a outro ecossistema

[Abrir fork ComfyUI](#)

A comparação com o OpenReel oficial apontou 18 commits à frente e 173 atrás, com alterações em editor, motion paths, partículas e ponte para fluxos de IA. O fork documenta integração com ComfyUI, mas sua base está significativamente defasada.

Decisão: estudar sua estratégia de integração; não escolhê-lo como substituto automaticamente superior ao OpenReel atual.



GitHub

3. `afatabe/openreel` e `zhangtsh5/aiVideo`

[afatabe/openreel](#) · [zhangtsh5/aiVideo](#)

A comparação identificou alterações expressivas: respectivamente, 159 e 149 commits à frente, ambos 173 atrás do OpenReel oficial. O segundo possui, entre outras alterações, módulos e testes para uma aplicação de imagem.

São fontes úteis para recuperar funcionalidades seletivas. Entretanto, sem executar regressões e verificar os conflitos com a arquitetura atual, não é defensável declarar qualquer um superior ao upstream.

4. `Relo-video/SynthCut` — um concorrente, não um fork

Este projeto surgiu na investigação fora da lista inicial. Ele utiliza um núcleo de edição em Node/TypeScript, comandos RPC validados por Zod, interface Electron/React, MCP, FFmpeg, Whisper e formato de timeline baseado em frames.

O aspecto mais interessante é sua decisão explícita de usar o mesmo núcleo de comandos para interface humana e agente, reduzindo risco de divergência de estado. A documentação informa 85 ferramentas MCP em um ponto e 94 em outro, evidenciando inconsistência de contagem. As interfaces de código existem, mas o comportamento final requer execução.



GitHub

Conclusão sobre os forks: encontrei implementações relevantes e não triviais, mas nenhum fork auditado pode ser declarado superior ao OpenReel oficial como base geral, nem superior ao Penpot oficial em estabilidade. A exceção é o `girafic/penpot`, que oferece capacidades de motion ausentes no Penpot original, em troca de riscos de compatibilidade e manutenção.

## Parte III — Análise arquitetural

A questão decisiva é escolher o proprietário do estado editável: quem conserva objetos, layers, animações, mídias, revisões e referências de projeto.

### As quatro estratégias principais

| Alternativa                                               | Benefício                                    | Custo ou risco determinante                                                                          |
|-----------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------|
| A. Modificar profundamente um editor                      | Muitas funcionalidades e UX já prontas       | Fork progressivamente incompatível com o projeto de origem                                           |
| B. Criar editor próprio com bibliotecas                   | Controle total de interface e contratos      | Implementação de seleção, ferramentas, timeline, persistência, undo, exportação e interoperabilidade |
| C. Reutilizar um núcleo de edição e adicionar adaptadores | Equilíbrio entre prontidão e extensibilidade | Necessita contratos explícitos entre engines e disciplina sobre propriedade do estado                |
| D. Unir aplicativos independentes sob uma janela          | Acesso rápido a capacidades maduras          | Cada aplicativo conserva seu documento; a experiência não é realmente unificada                      |

A opção C é a melhor hipótese arquitetural. Entretanto, há um detalhe fundamental: ela só é superior se as funcionalidades já existentes no editor escolhido continuarem funcionando sem exigir uma reescrita de suas estruturas internas.

## Um documento, três níveis de integração

Projeto Hermes

Manifesto versionado, ativos, revisões, proveniência e relações entre documentos

Objetos nativos

Camadas e keyframes editáveis

Referências vinculadas

Penpot, GLB e projetos externos

Ativos derivados

Frames, áudio, proxies e renders

Uma operação da IA modifica o objeto nativo quando existe suporte semântico. Caso contrário, opera na ferramenta proprietária do ativo e atualiza a referência e a revisão. O render derivado nunca substitui silenciosamente o original editável.

Essa estrutura permite diferentes graus de editabilidade:

| Nível     | Exemplo                                                                  | Garantia possível                                                                       |
|-----------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Nativo    | Texto, formas, vídeo e keyframes armazenados no mesmo documento OpenReel | Editar propriedades diretamente no editor principal                                     |
| Vinculado | Uma cena Blender ou design Penpot referenciado                           | Editar na aplicação de origem e atualizar a composição                                  |
| Derivado  | PNG, sequência de frames ou GLB exportado                                | Reproduzir/regerar o resultado; não recuperar necessariamente toda a semântica original |

Um manifesto não é um editor universal. Ele registra e organiza os ativos; só um modelo de objetos e operações compartilhado pode proporcionar edição sem perdas dentro de um mesmo canvas.

Assim, a estratégia correta é preservar os formatos nativos no início, adotar uma composição principal capaz de receber camadas de diferentes tipos e somente expandir um modelo

canônico próprio quando houver operações comuns que realmente precisem dessa abstração.

## Parte IV — Auditoria de código dos finalistas

### 1. OpenReel — principal candidato à composição central

OpenReel

TypeScript, React, MIT; arquitetura em monorepo, com editor web/desktop, core e agente.

[Repositório oficial](#)

Inspecionei os seguintes caminhos concretos:

| Arquivo ou módulo                      | Evidência encontrada                                                                                          |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <a href="#">types/project.ts</a>       | Project agrega timeline, textos, shapes, SVG, máscaras, composições motion, instâncias e estado de criação 3D |
| <a href="#">motion/types.ts</a>        | Camadas tipadas como text, shape, image, video, group, composition, particle, scene3d                         |
| <a href="#">motion-scene3d.ts</a>      | Construção de layer 3D com objetos, materiais, câmera, luzes e keyframes                                      |
| <a href="#">project-serializer.ts</a>  | Serialização, normalização e compatibilidade de versão de projeto, com schema 1.2.0                           |
| <a href="#">agent/headless-host.ts</a> | HeadlessHost, ActionExecutor, transações e rollback                                                           |
| <a href="#">agent/executor.ts</a>      | Ferramentas de agente executam ações sobre EditingHost                                                        |
| <a href="#">export-engine.ts</a>       | Exportação por WebCodecs/MediaBunny, gerenciamento de áudio, vídeo e GPU                                      |
| <a href="#">AGENT-CAPABILITIES.md</a>  | Catálogo gerado automaticamente descrevendo ferramentas e operações de edição                                 |

### O OpenReel resolveu o documento criativo universal?

Parcialmente, e isso é muito mais importante do que simplesmente reunir várias ferramentas na mesma interface.

O tipo Project possui uma única estrutura superior de persistência. Nela estão a timeline principal, as composições de motion e o estado de criação. Uma MotionComposition pode conter camadas de vídeo, vetores, tipografia, partículas e objetos scene3d.

Isso permite que parte substancial da criação, motion e vídeo permaneça realmente editável sob uma estrutura de projeto compartilhada.

Mas a análise também mostra limites:

- O projeto conserva submodelos diferentes, incluindo `timeline`, `motionCompositions` e `creation`. Isso não é uma única ontologia homogênea de objetos.
- O `HeadlessHost` executa edições sobre o estado em memória, mas delega renderização e trabalhos GPU a um `JobRunner` opcional. Portanto, headless editing não comprova headless rendering.
- O modelo de mídia admite referências e pistas de relink, com objetos `Blob` e `FileSystemFileHandle` que não são serializáveis como conteúdo persistente normal.
- Existem implementações e testes para composição 3D, mas não uma prova de que todo GLB, rig, animação ou shader sobreviva ao ciclo completo de importação, edição, salvamento e exportação.
- A documentação do agente apresenta uma contagem de 303 ferramentas em um catálogo gerado, enquanto outras referências registram números anteriores. A quantidade carregada em runtime permanece não verificada.
- Houve relato de falha por falta de memória durante exportação longa; o issue foi encerrado, mas seu encerramento não prova que o cenário atual esteja resolvido em todas as

configurações.  GitHub

Veredito: o OpenReel não deve ser tratado como universal sem ressalvas, mas já contém a abstração mais próxima da visão procurada entre os candidatos examinados. É prioritário executar seus seis testes de prova.

## 2. Penpot e o fork `girafic` — a melhor implementação vetorial animada encontrada

Penpot / Penpot Motion

Clojure/ClojureScript, renderização WASM, MPL-2.0.

[Código da branch de animação](#)

No fork, a implementação não cria um segundo arquivo de animação: a timeline vive em uma chave opcional `:timelines` da página Penpot, com tracks vinculados aos IDs das camadas.

Arquivos determinantes:

- [animation.cljc](#) — schema de keyframes e interpolação compartilhada pelo editor, viewer e exportador.
- [timelines.cljc](#) — conservação de animações ao duplicar, copiar e mover layers.
- [workspace/animation.cljs](#) — mudanças persistentes via undo e sincronização, com playhead transitório.

- [exports/animation.cljs](#) — exportação utilizando renderizador WASM e bibliotecas de mídia.
- [timelines\\_test.cljc](#) — testes de conservação de animação nas operações sobre camadas.

Esse desenho de dados oferece uma lição de engenharia valiosa: animações devem referenciar os mesmos IDs e objetos do editor, e a interpolação deve modificar a projeção visual, não sobrescrever permanentemente os valores de repouso.

O problema é o escopo. O Penpot Motion ainda não entrega um NLE audiovisual completo nem uma cena 3D editável. Acrescentar todas essas áreas ao núcleo Clojure/WASM transformaria o Hermes em mantenedor de um fork complexo que corre atrás de duas famílias distintas de software.

Veredito: melhor referência para preservar design vetorial e motion em um mesmo documento 2D; inadequado, no estado atual, como escolha imediata para todo o workstation.

### 3. SynthCut — a alternativa arquitetural mais interessante para vídeo e IA

A descoberta de [SynthCut](#) merece atenção porque contradiz a ideia de que precisaríamos necessariamente criar do zero o mecanismo de estado único entre agente e editor de vídeo.

Seu núcleo implementa:

- [types.ts](#) — modelo NLE não destrutivo, com tempos canônicos em frames.
- [rpc.ts](#) — contratos de operações definidos uma vez com Zod e compartilhados por WebSocket e MCP.
- [engine.ts](#) — engine, mídia, exportação FFmpeg, jobs, Whisper, edição e caches.
- [motion/render.ts](#) — composição Remotion renderizada para vídeo transparente ou opaco.

O modelo de projeto utiliza uma EDL (Edit Decision List). Isso é particularmente interessante para cortes, efeitos e áudio. Entretanto, a composição motion produzida em Remotion atravessa uma fronteira de renderização: o resultado inserido na timeline é uma mídia derivada, enquanto o código TSX continua sendo sua fonte autoral.

Assim, SynthCut é uma base forte para investigar um NLE operacional controlado por agentes, mas seu documento é menos abrangente para edição vetorial e 3D sem conversões.

Adicionalmente, utiliza GPL-3.0 e Remotion com licença própria, elevando a complexidade de distribuição. A documentação do projeto descreve essas obrigações explicitamente.

### 4. OpenCut — por que não é finalista principal

O repositório atual já possui `apps/web`, `apps/desktop`, `apps/api` e infraestrutura preparatória de mídia. Contudo, a inspeção de `crates/media` encontrou apenas arquivos de

preparação de builds FFmpeg, não o núcleo Rust completo anunciado.

O [plano oficial da reescrita](#) ainda lista como pendentes o core, API de edição, plugins, headless, MCP e parte da interface.

Veredito: não vale assumir risco de construir o Hermes sobre uma engine que ainda precisa concluir os próprios contratos centrais. Isso pode mudar quando houver releases e integração testável.

## 5. Babylon.js Editor e Graphite

O Babylon.js Editor tem uma coleção de handlers concretos para operações de agente e uma ponte HTTP com token e validações. Mas o ponto de ativação do MCP precisa ser confirmado numa build real. É uma alternativa séria ao Three.js Editor quando edição de cena por IA for central.

Já o Graphite é especialmente relevante por seu grafo procedural e edição não destrutiva. Entretanto, sua própria documentação coloca timeline de keyframes, dope sheet e curvas em desenvolvimento para o final de 2026. É um candidato a inspirar a arquitetura futura, não a resolver o prazo de entrega atual.



GitHub

+1

# Parte V — Comparação técnica e matriz de decisão

## 1. Matriz de capacidades

Legenda: Sim = há implementação identificada no código; Parcial = cobertura limitada ou fragmentada; Não comprovado = falta evidência suficiente; Planejado = consta como intenção ou roadmap.

| Capacidade                      | OpenReel | Penpot Motion | SynthCut  | OpenCut atual  |
|---------------------------------|----------|---------------|-----------|----------------|
| Edição visual 2D                | Sim      | Sim           | Parcial   | Parcial        |
| Ferramentas vetoriais avançadas | Parcial  | Sim           | Limitadas | Não comprovado |

| Capacidade                         | OpenReel               | Penpot Motion               | SynthCut                             | OpenCut atual                 |
|------------------------------------|------------------------|-----------------------------|--------------------------------------|-------------------------------|
| Timeline audiovisual               | Sim                    | Parcial                     | Sim                                  | Parcial                       |
| Keyframes editáveis visualmente    | Sim                    | Sim                         | Sim                                  | Parcial                       |
| Motion graphics                    | Sim                    | Sim, 2D                     | Via Remotion e overlays              | Não comprovado                |
| Objetos 3D estruturados            | Sim, com limitações    | Não                         | Não identificado como recurso nativo | Não comprovado                |
| IA editando o estado do projeto    | Sim                    | Sim, via API/plugin no fork | Sim                                  | Planejado                     |
| API para edição headless           | Sim                    | Parcial, via integrações    | Sim                                  | Planejado                     |
| Renderização headless independente | Parcial                | Não comprovado              | Via FFmpeg/Remotion                  | Planejado                     |
| Salvar/reabrir estrutura editável  | Implementado no código | Implementado no código      | Implementado no código               | Não comprovado no novo núcleo |
| Paridade preview/export            | Não comprovada         | Não comprovada              | Não comprovada                       | Não comprovada                |

Aqui, Sim não equivale a teste de aceitação aprovado. Indica que encontrei estruturas e funções concretas. A classificação de produção depende de execução e validação.

## 2. Pontuação ponderada

Os critérios mantêm exatamente os pesos solicitados. As notas são estimativas de adequação fundamentadas na auditoria estática, não pontuações de conformidade certificada.

Pesos originaisEntrega rápidaLongo prazo

1º

Híbrido: OpenReel + adaptadores

7.15

2º

Fork profundo de OpenReel

6.85

3º

Federação de aplicativos

6.30

4º

SynthCut + extensões

5.80

5º

Editor próprio React

5.80

6º

Penpot Motion como base

5.20

7º

Graphite como base

4.70

8º

OpenCut reescrito

2.60

Notas provisórias de 0 a 10. Altere a prioridade para observar a sensibilidade da classificação.

Notas atribuídas por critério — avaliação original

| Opção | Funcionalidades | Integração IA | Documento | Velocidade | Arquitetura | Manut |
|-------|-----------------|---------------|-----------|------------|-------------|-------|
|-------|-----------------|---------------|-----------|------------|-------------|-------|

|                                          | 25% | 15% | 20% | 15% | 10% | 10% |
|------------------------------------------|-----|-----|-----|-----|-----|-----|
| Híbrido:<br>OpenReel<br>+<br>adaptadores | 7   | 8   | 7   | 7   | 8   | 6   |
| Fork<br>profundo<br>de<br>OpenReel       | 7   | 8   | 6   | 8   | 6   | 5   |
| Federação<br>de<br>aplicativos           | 8   | 7   | 3   | 7   | 6   | 7   |
| Editor<br>próprio<br>React               | 3   | 8   | 8   | 2   | 9   | 6   |
| SynthCut +<br>extensões                  | 6   | 8   | 4   | 6   | 7   | 5   |
| Penpot<br>Motion<br>como base            | 5   | 7   | 5   | 4   | 6   | 4   |
| Graphite<br>como base                    | 4   | 4   | 4   | 3   | 8   | 6   |
| OpenCut<br>reescrito                     | 2   | 1   | 2   | 2   | 5   | 3   |

## Justificativa das notas

Híbrido com OpenReel: recebe nota alta por reutilizar timeline, motion, 3D parcial e comandos para IA que já existem em código. Perde pontos de manutenção pela necessidade de controlar versões, integrações e divergências entre motores.

Fork profundo de OpenReel: pontua melhor em velocidade inicial porque aproveita a interface inteira. Perde pontos no longo prazo pela dependência direta dos detalhes internos do editor e pelo risco crescente de conflito com upstream.

Federação de aplicativos: possui elevada cobertura funcional agregada; Penpot, Blender e um NLE são poderosos individualmente. Sua nota baixa de documento reflete a incapacidade de garantir editabilidade unificada apenas exibindo os aplicativos juntos.

Editor próprio: recebe notas altas para extensibilidade potencial, mas baixas em prontidão. Seu custo principal é reconstruir funcionalidades não diferenciadoras que já existem em outros projetos.

SynthCut: sua RPC comum a UI/MCP é uma decisão arquitetural especialmente forte. Perde pontos pela cobertura menor de design vetorial e 3D nativo, além da licença GPL e dependência opcional de Remotion.

Penpot Motion: já implementa uma integração interessante entre vetores, keyframes e colaboração, mas não um NLE ou ambiente 3D. A defasagem de centenas de commits em relação ao upstream e a natureza experimental justificam penalização de manutenção.

Graphite: possui arquitetura procedural promissora, mas precisa concluir recursos essenciais de animação antes de competir como estação audiovisual completa.

OpenCut: o potencial da reescrita é relevante, mas não pode substituir funcionalidades implementadas e verificadas hoje.

## **Análise de sensibilidade**

Prioridade: entrega rápida

**7,20/10**

Fork/adaptação do OpenReel

Vantagem pequena sobre a estratégia híbrida (7,10), porque reduz o esforço inicial de integração.

Prioridade: longo prazo

**7,30/10**

Híbrido com adaptadores

Vantagem estreita sobre editor próprio (7,25), especialmente pela extensibilidade sem descartar o código existente.

A diferença entre algumas alternativas é pequena demais para justificar uma decisão irreversível. Com incerteza razoável de aproximadamente um ponto por critério, o primeiro lugar pode mudar após testes reais.

A recomendação é robusta em sua direção, mas não na seleção definitiva da implementação: reutilizar um núcleo de edição existente agora e conservar a opção de extrair ou substituir componentes depois.

### 3. Prova prática executada

Consegui executar um teste independente de pipeline audiovisual com FFmpeg 7.1.5 em ambiente isolado.

O teste criou dois trechos de vídeo, aplicou texto com posição variável, concatenou as cenas, adicionou áudio e exportou o MP4. O `ffprobe` confirmou:

- Vídeo H.264 a 640 × 360 e 30 fps.
- Áudio AAC.
- Duração final de 6 segundos.
- Arquivo MP4 gerado com sucesso e hash SHA-256 registrado.

Prova de codificação audiovisual

Teste independente de FFmpeg; não é uma execução do OpenReel, Penpot ou SynthCut.

Abrir vídeo de teste · Frame PNG

Os repositórios dos editores não puderam ser clonados no ambiente de execução disponível porque a conexão de saída para GitHub falhou na resolução de DNS. Portanto, não executei builds, testes de interface ou integração desses candidatos.

A situação dos seis testes solicitados é a seguinte:

| Teste                                        | Resultado nesta investigação                                       |
|----------------------------------------------|--------------------------------------------------------------------|
| 1. Arte 2D editável, salvar/reabrir/exportar | Não executado no editor                                            |
| 2. Keyframes e MP4 preservando projeto       | Não executado no editor                                            |
| 3. Cortes de vídeo, sobreposição e áudio     | Pipeline FFmpeg básico comprovado isoladamente; editor não testado |
| 4. Cena 3D na composição                     | Não executado                                                      |
| 5. IA editando o mesmo projeto               | Implementação identificada estaticamente; sem E2E                  |
| 6. Integridade e paridade preview/export     | Não executado                                                      |

Nenhum dos seis testes de aceitação está aprovado integralmente. Essa é a principal limitação antes de uma decisão definitiva sobre o editor-base.

### Parte VI — Recomendação técnica final

# 1. Qual projeto utilizar como base?

OpenReel é o principal candidato ao editor-base.

Ele possui a combinação mais próxima do objetivo: interface visual, timeline, motion graphics, edição audiovisual, camadas 3D e API de operações para agentes.

Mas minha escolha de arquitetura não é transformar todo o Hermes em um fork monolítico do OpenReel.

O desenho recomendado é:

Arquitetura-alvo: engines especializados, mas execução e autoridade centralizadas no Hermes.

O OpenReel seria o ambiente principal para compor e editar conteúdos. As outras ferramentas teriam papéis especializados, sem substituir as camadas de segurança ou o sistema de execução do Hermes.

# 2. Existe algum fork superior?

Há dois destaques distintos.

O `girafic/penpot` é superior ao Penpot original especificamente por implementar motion 2D integrado às camadas vetoriais. Porém, ainda é experimental e significativamente divergente do upstream.

Os forks do OpenReel auditados possuem implementações úteis, incluindo integração com ComfyUI e melhorias em ferramentas visuais, mas não há prova suficiente para considerá-los superiores ao OpenReel atual como plataforma geral.

# 3. Quais bibliotecas e motores adicionar?

Recomendação por prioridade:

| Prioridade | Tecnologia                     | Razão                                                       |
|------------|--------------------------------|-------------------------------------------------------------|
| Essencial  | OpenReel core, UI e agent      | Reduzir construção redundante do editor                     |
| Essencial  | FFmpeg/ffprobe                 | Processamento, codificação, validação e exportação          |
| Alta       | MediaBunny/WebCodecs           | Trabalhar com mídia e preview no navegador                  |
| Alta       | Three.js existente no OpenReel | Ampliar edição e renderização 3D sem segundo motor imediato |

| Prioridade           | Tecnologia               | Razão                                                        |
|----------------------|--------------------------|--------------------------------------------------------------|
| Alta, sob demanda    | Penpot/MCP               | Design vetorial avançado e colaboração                       |
| Média                | Blender Python/headless  | Rigging, modelagem e renderização 3D complexa                |
| Média, alternativa   | Babylon.js Editor        | Avaliar editor 3D operacional por MCP                        |
| Média, especializada | Motion Canvas ou Revideo | Animações programáticas complementares                       |
| Condicional          | Remotion                 | Conteúdos React complexos, somente depois do gate de licença |
| Pesquisa             | Graphite                 | Grafos de operações e edição não destrutiva                  |

Não adicionaria todas essas ferramentas no primeiro protótipo. Em particular, não introduziria Penpot e Babylon ao mesmo tempo em que tenta estabilizar o OpenReel.

## 4. Qual a melhor integração com IA?

A IA deve operar comandos tipados sobre o modelo de projeto, não automatizar cliques como mecanismo principal.

Exemplo de contrato conceitual:

```
{
  "operation": "creative.update_property",
  "project_ref": "project-123",
  "expected_revision": 42,
  "target_id": "headline-001",
  "property": "fontSize",
  "value": 64,
  "effect_scope": "project_document"
}
```

Fluxo recomendado:

1. O agente lê o projeto e descobre objetos, capacidades e revisões.
2. Converte a intenção do usuário em operações verificáveis.
3. O Router/Policy existente autoriza os efeitos apropriados.
4. Um adapter transforma o comando Hermes em uma operação OpenReel, Penpot, Blender ou outra engine.

5. A engine modifica o mesmo documento usado pela interface humana.
6. O Hermes faz readback, valida revisões e registra o resultado no ArtifactStore/Journal.
7. O Experience Compiler pode aprender a operação depois de execução realmente verificada.

A IA não pode sobrescrever a edição humana com uma revisão antiga. Os adapters devem exigir controle de concorrência, com `expected_revision`, IDs persistentes e reconciliação de conflitos.

Laya/System-1 pode auxiliar na seleção de operações e detecção de oportunidades de reutilização, mas não deve conceder permissões, inventar provas de sucesso ou promover capacidades por confiança inferida.

## 5. Como preservar o projeto editável?

Recomendo separar duas unidades:

Documento de composição principal: inicialmente o formato nativo OpenReel, que conserva timeline, camadas, keyframes e elementos que seu editor entende.

Pacote Hermes Creative Project: um manifesto versionado com referências aos projetos nativos adicionais, ativos, versões de ferramentas, revisões, hashes e saídas.

Um projeto poderia possuir esta estrutura conceitual:

```
Meu-Projeto.hermes/  
├─ project.hermes.json  
├─ edit/  
│   └─ openreel.project.json  
├─ sources/  
│   ├── penpot/  
│   │   └─ design-reference.json  
│   ├── three/  
│   │   └─ scene.json  
│   └─ blender/  
│       └─ source.blend  
├─ assets/  
│   ├── logo.svg  
│   ├── produto.glb  
│   ├── gravacao.mp4  
│   └─ musica.wav  
├─ derived/  
│   ├── scene3d-preview.webm  
│   └─ proxy-video.mp4  
└─ exports/
```

└─ arte.png  
└─ composicao.mp4

Isso não exigiria que todos os formatos fossem convertidos para um JSON genérico de perdas desconhecidas.

Regra essencial: um objeto só pode ser considerado nativamente editável na composição principal quando sua engine e seu serializer suportarem explicitamente todas as propriedades relevantes. Caso contrário, preservar o arquivo original e o vínculo é mais correto do que simular editabilidade.

Para interoperabilidade de timeline, OpenTimelineIO pode servir como adapter de intercâmbio, não como schema-mestre. O mesmo vale para SVG e glTF: são formatos de intercâmbio em suas áreas, não substitutos universais dos formatos de projeto das aplicações.

## 6. Quais funcionalidades faltariam desenvolver?

| Já existe em projetos auditados       | Lacuna específica do Hermes                                    |
|---------------------------------------|----------------------------------------------------------------|
| Canvas, layers e timeline no OpenReel | Integrar a superfície visual ao BrowserTask/Electron existente |
| Comandos de edição para agentes       | Adapter tipado submetido ao Control Plane                      |
| Serialização de projetos              | Revisão concorrente, fingerprints, proveniência e manifest     |
| Motion e camadas 3D                   | Verificar e ampliar formatos e propriedades compatíveis        |
| Renderização e exportação             | Export job supervisionado e prova de saída real                |
| Integração MCP em outros editores     | Escopo de autorização, sessão, verificação e rollback          |
| FFmpeg, Whisper e outras engines      | Descoberta, instalação consentida, health e processos isolados |
| Experience Compiler                   | Provas empíricas de operações criativas antes do reuso         |

A maior lacuna não parece ser desenhar botões ou recriar uma timeline. É construir uma ligação segura e verificável entre o documento criativo, a interface e o runtime do Hermes.

## 7. Qual o caminho com menos trabalho para obter resultados reais?

Usar inicialmente o OpenReel como aplicativo completo, sem reescrever sua interface.

Primeiro produzir artes e vídeos dentro dele. Depois adicionar uma ponte mínima para o Hermes, reaproveitando o agente e os comandos existentes.

A alternativa SynthCut deve ser mantida como prova concorrente para edição de vídeo por IA, especialmente se a exportação do OpenReel apresentar problemas persistentes.

Para vídeos programáticos com controle fino de código, Revideo ou Motion Canvas podem produzir resultados independentemente enquanto a integração visual evolui. Não há razão para bloqueá-los até a conclusão do workstation.

## **8. Qual arquitetura tem melhor potencial de longo prazo?**

Uma arquitetura híbrida orientada a documentos, adapters tipados e múltiplos renderizadores.

Mas não uma coleção desconectada de aplicativos em abas.

É necessário haver uma engine proprietária do estado principal da composição, um sistema de identidade/revisão dos objetos e contratos explícitos para as operações delegadas aos subsistemas especializados.

## **9. O que não vale a pena desenvolver?**

Não desenvolveria de início:

- Outro Agent Runtime, TaskRun, sistema de permissões ou Experience Compiler.
- Uma timeline NLE básica do zero.
- Um motor próprio de codificação e transcodificação.
- Um editor de curvas Bézier geral.
- Uma engine WebGL/WebGPU 3D inteira.
- Um algoritmo próprio de transcrição ou segmentação de vídeo.
- Um protocolo MCP novo.
- Um formato universal que pretenda converter sem perdas Penpot, Blender, SVG, GLB, React e NLE sem provas de interoperabilidade.

Esses seriam investimentos de engenharia grandes, pouco diferenciadores e, em vários casos, duplicações diretas.

## **10. Quais hipóteses continuam abertas?**

As quatro mais importantes para escolher o editor-base definitivamente são:

1. A persistência do OpenReel realmente conserva todas as propriedades de motion, texto, vídeo e objetos 3D após salvar e reabrir?

2. A IA e o humano conseguem editar o mesmo projeto sem inconsistência, conflitos silenciosos, operações fora da revisão esperada ou perda de undo?
3. O render final corresponde à pré-visualização, especialmente em 3D, transparências, shaders, fontes e clips de longa duração?
4. O editor web pode ser incorporado ao Hermes Electron sem duplicar sessões, comprometer sandbox, expor APIs privilegiadas ou criar um segundo supervisor de processos?

Se o OpenReel falhar materialmente nos testes 1–3, a escolha de editor-base deve ser reaberta. Se falhar apenas no quarto, a integração como aplicativo local isolado pode continuar viável.

## Papel das linguagens na arquitetura

O Hermes já utiliza React/Electron e Python. Não há justificativa suficiente para uma reescrita completa do produto.

| Subsistema                       | Tecnologia preferencial            | Motivo                                                    |
|----------------------------------|------------------------------------|-----------------------------------------------------------|
| Interface visual                 | TypeScript + React                 | Reutilização direta do Desktop e dos editores candidatos  |
| Estado do documento              | TypeScript com schemas versionados | Tipagem de objetos, operações, revisões e serialização    |
| Contratos Hermes                 | Python + validação tipada          | Reutilização de TaskRun, Policy, Verifier e ArtifactStore |
| Renderização 2D                  | SVG/Canvas/GPU conforme tipo       | SVG para estruturas vetoriais; Canvas/GPU para composição |
| Renderização 3D                  | Three.js/WebGL/WebGPU              | Bibliotecas maduras e compatíveis com React               |
| Shaders customizados             | GLSL/WGSL                          | Efeitos GPU especializados, sem reinventar o motor        |
| Vídeo e áudio                    | FFmpeg, MediaBunny, WebCodecs      | Ferramentas especializadas já disponíveis                 |
| Inferência local                 | Python e runtimes ONNX/CUDA        | Ecosistema amplo de modelos                               |
| Processamento de alto desempenho | Rust/C++/WASM conforme benchmark   | Introduzir somente onde houver gargalo demonstrado        |
| Plugins/adapters                 | Processos isolados com API/MCP/CLI | Menor acoplamento e contratos de efeito verificáveis      |

Rust é uma opção importante para futuras operações pesadas, mas não uma obrigação arquitetural imediata. A maturidade do MLT/FFmpeg e a existência de um núcleo TypeScript no OpenReel tornam pouco justificável começar pela criação de uma nova engine Rust.

## Licenciamento e distribuição

| Família de licença     | Implicação para o projeto                                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| MIT / BSD / Apache-2.0 | Geralmente mais simples para incorporar, modificar e distribuir; preservar avisos e requisitos específicos                                        |
| MPL-2.0                | Obrigações de disponibilizar o código-fonte de arquivos cobertos modificados quando distribuídos, segundo os termos aplicáveis                    |
| LGPL                   | Possibilita usos com componentes separados, mas exige cumprir condições específicas de redistribuição, substituição/relink e avisos               |
| GPL                    | Pode impor reciprocidade ao trabalho combinado distribuído; integração por processo separado requer análise concreta da relação entre componentes |
| AGPL                   | Acrescenta obrigações relevantes para disponibilização de software modificado a usuários por rede                                                 |
| Licenças próprias      | Verificar autorização para distribuição, serviços, automação e derivados                                                                          |

Para o Hermes, os pontos mais sensíveis são Remotion, Penpot, builds de FFmpeg, bibliotecas MediaBunny, módulos de Theatre.js, modelos de IA e SDKs com licença comercial.

Ter um repositório MIT não significa que todos os binários, pesos, fontes, codecs e dependências distribuídos também estejam sob MIT.

## Parte VII — Plano de implementação

### Plano A — Resultado imediato

Objetivo: criar e exportar artes, motion graphics e vídeos reais o mais rapidamente possível, sem esperar pelo nascimento de um editor universal.

#### 1. Preparar uma execução isolada do OpenReel

Dependências: Node, pnpm, build pinada, Chromium/Electron e codecs suportados.

Usar inicialmente `apps/web` e `packages/core` do OpenReel, sem incorporar uma segunda aplicação Electron ao Hermes.

Aceite: editor inicia, importa arquivos reais, cria projeto, salva e reabre.

## 2. Executar os testes de design e motion

Criar um material de comunicação com texto, grupos, vetores e imagem; animar ao menos três propriedades com keyframes.

Aceite: modificações continuam editáveis depois do reload e a animação exporta MP4 com frames verificáveis.

## 3. Executar um caso real de edição audiovisual

Importar vídeo e música, realizar cortes, adicionar um título animado e exportar vídeo vertical.

Aceite: duração, dimensões, áudio e sequência temporal batem com a edição; validar com ffprobe e inspeção de frames.

## 4. Verificar edição 3D e agente

Adicionar uma camada de cena 3D; modificar propriedades manualmente e por operações do pacote `agent`.

Aceite: a estrutura persiste e o agente não perde as alterações humanas.

## 5. Decidir integração mínima

Se os testes passarem, ligar o editor ao Hermes por um adapter estreito, inicialmente em ambiente controlado.

Aceite: o Hermes solicita uma operação, o mesmo projeto muda, o resultado é relido e um artefato verificável é registrado.

Dependência crítica: as etapas executadas dentro do repositório Hermes continuam sujeitas a H-079, CI, segurança e autorização. As provas isoladas de software externo podem ocorrer antes, sem alterar branches ou runtime do Hermes.

# Plano B — Evolução para um Creative Workstation unificado

| Fase           | Objetivo                                                           | Subsistemas principais                                                  | Critério de conclusão                                         |
|----------------|--------------------------------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------|
| CW-01 revisada | Revalidar baseline e a decisão com provas OpenReel/SynthCut/Penpot | workstation/creative-workstation/, Source Matrix, código dos candidatos | Evidência, SHAs, licenças e gates atuais                      |
| CW-02          | Adapter de aplicação criativa com lifecycle supervisionado         | Resolver, ProcessRegistry, capabilities, health                         | Instalação consentida health real, restart/suporte isolamento |

| Fase  | Objetivo                                         | Subsistemas principais                                   | Critério de conclusão                                    |
|-------|--------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------|
| CW-03 | Primeira composição nativa editável e exportável | OpenReel core, agent, Hermes BrowserTask, artifacts      | E2E 2D + motion + + salvamento                           |
| CW-04 | Estado único entre agente e humano               | Comandos tipados, revision checks, journal               | Testes de concorrência e undo e readback                 |
| CW-05 | Pipeline robusto de exportação                   | FFmpeg, MediaBunny, render jobs, verifier                | Preview/export dentro tolerâncias definidas              |
| CW-06 | Integração 3D                                    | OpenReel scene3d, Three.js, Blender opcional             | GLB e cenas reeditáveis material/câmera/key              |
| CW-07 | Design vetorial especializado                    | Penpot/MCP e importadores SVG                            | Revisões humanas preservadas e assets vinculados         |
| CW-08 | Projeto portátil                                 | Manifest, assets, hashes, schemas e migrações            | Reabrir em outro ambiente sem perder recursos declarados |
| CW-09 | IA criativa especializada                        | SAM 2, Whisper, rembg e adapters                         | Operações geram assets e parâmetros editáveis            |
| CW-10 | Aprendizagem operacional                         | Experience Compiler, OperationalCapabilityRegistry, Laya | Reutilização após verificação empírica replay válido     |

A sequência deve ser implementada em unidades pequenas, cada uma com teste positivo, teste negativo, critérios de parada e rollback. A versão do OpenReel — ou de qualquer engine externa — precisa ficar pinada e ser atualizada por mudanças controladas.

## Portões antes de uma adoção definitiva

| Portão                  | Pergunta de aprovação                                              | Se falhar                                                |
|-------------------------|--------------------------------------------------------------------|----------------------------------------------------------|
| G1 — Documento          | Salvar/reabrir conserva objetos, keyframes e mídias?               | Não adotar como documento principal                      |
| G2 — Colaboração com IA | Interface e agente alteram o mesmo estado com concorrência segura? | Corrigir adapter ou trocar núcleo                        |
| G3 — Exportação         | Render final preserva conteúdo e propriedades importantes?         | Avaliar FFmpeg/MLT e pipeline alternativo                |
| G4 — 3D                 | Cena, materiais, animação e GLB continuam editáveis?               | Usar 3D vinculado, não conversão supostamente sem perdas |

| Portão                     | Pergunta de aprovação                              | Se falhar                           |
|----------------------------|----------------------------------------------------|-------------------------------------|
| G5 — Electron              | Funciona sob BrowserTask e isolamento existentes?  | Manter processo externo isolado     |
| G6 — Licenças e manutenção | Dependências podem ser distribuídas e atualizadas? | Excluir componente ou trocar engine |

## Decisão recomendada

Para entregar resultados imediatamente

Usar OpenReel completo, preservar sua interface e provar o fluxo de criação, edição, motion e exportação antes de construir qualquer UI própria.

Para construir o produto definitivo

Evoluir para uma arquitetura híbrida com OpenReel como núcleo inicial de composição, motores especializados desacoplados, manifesto de projeto, operações tipadas, revisões e autoridade centralizadas no Hermes Work.

Antes de assumir compromisso de longo prazo

Executar a bateria E2E dos três candidatos críticos: OpenReel, SynthCut e Penpot Motion. A integridade do documento e a paridade entre preview/exportação devem prevalecer sobre a quantidade de funcionalidades anunciadas.

A conclusão mais importante desta investigação é que não precisamos escolher entre construir um editor do zero e modificar completamente o Penpot. O OpenReel já oferece grande parte da combinação procurada, o SynthCut demonstra outra arquitetura de edição controlada por IA, e o fork Penpot Motion comprova que é viável incorporar animação no documento vetorial sem separar design de motion.

Minha recomendação é começar pelo OpenReel, mas não tornar todo o Hermes dependente da arquitetura interna dele. Validar primeiro, adaptar o mínimo necessário e preservar a possibilidade de substituir subsistemas especializados é a estratégia que melhor equilibra velocidade, editabilidade e manutenção.

Escopo e integridade: a investigação envolveu documentação, metadados, comparação de branches e forks, leitura de implementações reais e uma prova isolada do FFmpeg. Não houve alterações nos repositórios, branches ou documentação do Hermes. A auditoria é suficiente para priorizar protótipos e orientar a arquitetura, mas a seleção final do editor-base permanece condicionada aos testes de aceitação que não puderam ser executados neste ambiente.